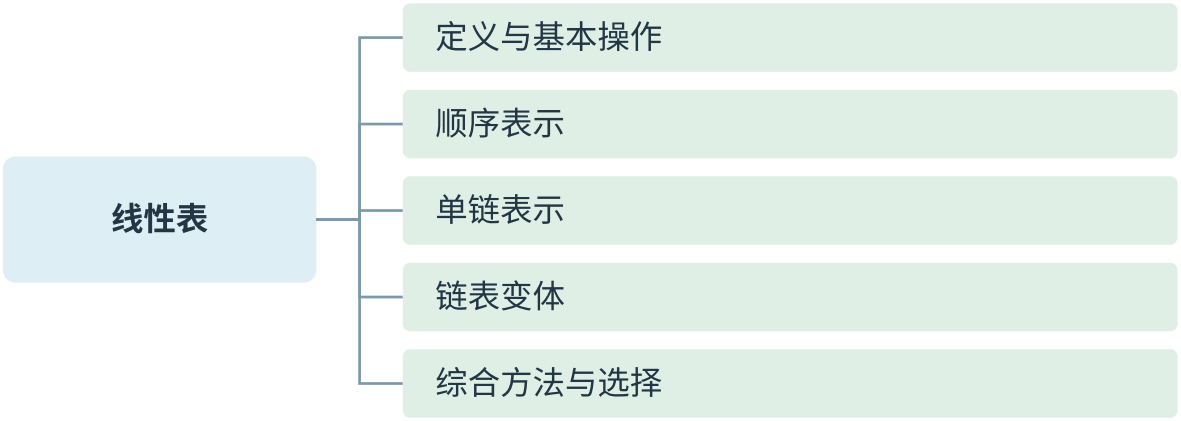
线性表 2

章节脉络



同一个线性表可以放在连续数组中，也可以由分散的结点连接起来。前者依靠数组下标定位，插入和删除常要搬移元素；后者把先后关系放在链接中，定位常要逐结点进行。分析操作复杂度时，必须把“找到位置”和“修改结构”分别计算。

最小反例·容量不等于表长

容量为 100 的数组可以表示空表，此时 length=0；也可只保存一个元素。仅给数组上界不能推断有效元素数。插入判断必须同时检查位序是否合法与是否尚有空位。

2.1 位序、下标和基本操作

线性表是具有相同数据类型的 n 个元素的有限序列。$n=0$ 时为空表。非空表的第一个元素没有直接前驱，最后一个没有直接后继，其余元素各有一个直接前驱和一个直接后继。这里的“相同类型”不要求元素的值互不相同。

长度为 n 的表，已有元素的位序是 $1\sim n$；在第 i 个位置插入新元素，合法范围是 $1\le i\le n+1$。最后一个位置表示追加。若用 `A[0..n-1]` 保存，位序 i 对应下标 $i-1$。查找失败的信号不能与下标 0 混淆，例如按位序返回 0 表示失败，或按下标返回 -1 表示失败。

本章约定单链表有头结点 `L`，首元结点为 `L->next`。头结点用于统一第一个位置的插入与删除，不保存有效表元素；**头指针**是指向它的指针变量，二者不是同一对象。无头结点的版本也可以实现相同运算，但修改第一个结点时必须能够修改头指针。

2.2 顺序表的寻址、插入和删除

若每个元素占 w 个地址单位，首元素地址为 b，则第 i 个元素地址为 $b+(i-1)w$。计算地址只需常数次算术运算，故可随机存取。“随机”指任意位置可以直接定位，与随机数无关。按值查找仍要比较元素，未排序时通常需要 $O(n)$ 时间。

在下标 k 处插入，要把 `A[k..n-1]` 各后移一位；必须**从后向前**移动，否则后一个元素可能已被前一个覆盖。删除下标 k 的元素，则把 `A[k+1..n-1]` 各前移一位，按从前向后的顺序执行。

```
insert(A, n, capacity, k, x):
    if k<0 or k>n or n==capacity: return false
    for j=n down to k+1: A[j]=A[j-1]
    A[k]=x
    n=n+1
    return true
```

不变量·搬移保护未读区

插入向后搬移时，从后往前处理能确保源元素尚未被覆盖；删除向前搬移则从前往后读。记住需要保护的未读区，就能由目标下标推回正确循环方向。

插入移动 $n-k$ 个元素，追加时为 0；删除移动 $n-k-1$ 个元素，删除末元素时为 0。等概率在 $n+1$ 个插入位置选择时，平均移动 $n/2$ 个；等概率删除 n 个已有元素时，平均移动 $(n-1)/2$ 个。平均数的分母不同，来自合法位置数不同。

2.2.1 例 2-1　没有给存储方式时能否确定移动数

例 2.2.1 (配套例题)

〖2.3-012〗在线性表 $a_0, a_1, \ldots, a_{100}$ 中，删除元素 a_{50} 需要移动（　）个元素。

A. 0　B. 50　C. 51　D. 0 或 50

解析。题目没有限定存储方式。顺序表中，后面的 $a_{51} \sim a_{100}$ 共 50 个元素前移；链表中改变链接即可，无需搬移其他结点的数据，故答案为 **D**。删除前若还要从头寻找该结点，链表访问时间仍可能是 $O(n)$；“不移动数据” 不等于 “全过程常数时间”。

静态数组的容量固定，不意味着表长固定。动态数组可以在空间不足时申请更大的连续区并搬移旧元素，但单次扩容可能需要 $O(n)$ 时间；分析某一次插入时不能忽略这次复制。

2.3 顺序表的完整状态与核心实现

顺序表保存数据元素以及当前表长；动态分配版本还需记录容量。静态容量与表长是不同概念：容量固定时，表长仍可在 0 至容量之间变化。代码沿用教材的类型名称，`ElemType` 表示元素类型，本章示例取整型。

```
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
```

静态类型使用宏 `MaxSize` 作数组容量，后文动态类型以成员 `MaxSize` 记容量。静态与动态两组类型示例独立使用；编译验证时分别组织为独立单元。

2.3.1 初始化、查找与随机访问

静态顺序表初始化令 `length=0`。`InitList(SqList &L)`中的引用参数使函数能修改调用者的表长。动态顺序表还需申请连续空间，申请成功后设定表长和容量；销毁时释放该空间。

```cpp
void InitList(SqList &L) { L.length = 0; }
ElemType GetElem(SqList L, int i) {
    return L.data[i - 1];  // 前提: 1 <= i <= L.length
}
int LocateElem(SqList L, ElemType e) {
    for (int i = 0; i < L.length; ++i)
        if (L.data[i] == e) return i + 1;
    return 0;
}
```

辨析·失败不能改状态

先检查位序和容量，再搬移、赋值和改表长。删除结果 e 只在成功时有新值。

按位查找先把位序换成数组下标，时间 $O(1)$；调用前须保证位序合法。按值查找从前向后扫描，成功返回首次相等元素的位序，失败返回 0，最坏 $O(n)$。本章 ElemType 取整型，可直接用 `==`；若元素为结构类型，实际代码应定义逐字段或按关键字的判等函数。若成功位置等概率，平均比较 $(n+1)/2$ 次；有序顺序表可采用折半查找，具体算法在查找章展开。

2.3.2 插入与删除的循环方向

插入合法范围是 $1 \le i \le n+1$，还要保证有剩余容量。将从位序 i 开始的后缀从后向前搬移；删除合法范围是 $1 \le i \le n$，先保存被删元素，再将后缀从前向后搬移。

```cpp
bool ListInsert(SqList &L, int i, ElemType e) {
    if (i < 1 || i > L.length + 1) return false;
    if (L.length >= MaxSize) return false;
    for (int j = L.length; j >= i; --j)
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = e;
    ++L.length;
    return true;
}
bool ListDelete(SqList &L, int i, ElemType &e) {
    if (i < 1 || i > L.length) return false;
    e = L.data[i - 1];
    for (int j = i; j < L.length; ++j)
        L.data[j - 1] = L.data[j];
    --L.length;
    return true;
}
```

插入时先搬后面的元素，才能保存尚未读取的旧后缀；若从前向后执行 `data[j+1]=data[j]`，会反复复制同一个被覆盖的值。删除时源位置比目标位置大，从前向后搬移恰好保护未读元素。失败操作不修改表长，删除输出参数只在成功时有效。

2.3.3 动态分配与扩容

动态顺序表仍要求整个数组连续，只是容量可以重建。扩容不是在原地址后方一定能再接一段；需要取得新连续区，再搬移已有元素。下面保留教材的 `SeqList`、`data`、`MaxSize`、`length` 命名。`InitSize=100` 是王道 2027 教材中的示例初始容量；课件演示取 10，两者采用相同的存储结构和接口，实际值可以按需求设定。

```c
#define InitSize 100
typedef struct {
    ElemType *data;
    int MaxSize;
    int length;
} SeqList;
void InitList(SeqList &L) {
    L.data = (ElemType *)malloc(InitSize * sizeof(ElemType));
    L.length = 0;
    L.MaxSize = InitSize;
}
void IncreaseSize(SeqList &L, int len) {
    ElemType *p = L.data;
    L.data = (ElemType *)malloc(
        (L.MaxSize + len) * sizeof(ElemType));
    for (int i = 0; i < L.length; ++i)
        L.data[i] = p[i];
    L.MaxSize += len;
    free(p);
}
```

辨析·动态仍然连续

SeqList 动态区仍是一个连续数组。扩容先保住新旧地址，复制成功后再释放旧区，旧元素地址会变化。

这两段保留教材的 void 接口，以申请成功为前提；扩容还要求 len 为正，申请长度能由机器整数和地址空间表示。实际工程应先把 malloc 结果存入临时指针 q，检查非空后再替换 data；否则直接覆盖 data 可能在申请失败时丢失原地址。每次扩容搬移 n 个元素，时间 $O(n)$，新旧区同时存在时峰值临时空间也为 $O(n)$。`malloc/free` 成对使用；清空令长度为 0，销毁则释放数组并撤销有效指针、长度和容量。动态数组支持随机存取，动态分配本身不等于链式存储。

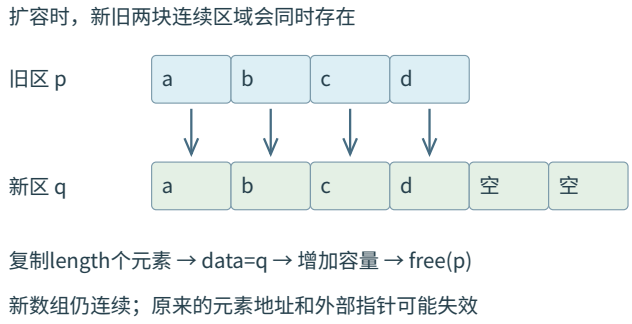


Figure 2.1: 动态顺序表扩容时复制有效元素并释放旧区

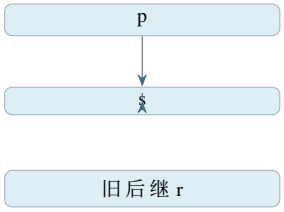
2.4 单链表：先保留哪条链接

本章采用教材的结点与头指针类型，申请与释放使用 `malloc/free`；对应代码包含 `<stdlib.h>`。

```c
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
bool InitList(LinkList &L) {
    L = (LNode *)malloc(sizeof(LNode));
    if (L == NULL) return false;
    L->next = NULL;
    return true;
}
```

最小反例·两句交换会丢链

p 原后继为 r。若先写 p->next=s，再写 s->next=p->next，第二句读到的是 s，形成自环，r 不再可从 p 到达。先暂存旧链接，是所有原地指针改写的基本纪律。



错误顺序使 s 自环，旧后继 r 不可达

结点由数据域与后继指针域构成。不同结点不要求物理相邻；同一个结构体结点所含的各域作为一个整体分配，因此不能把“结点之间地址分散”误说成“一个结点内部各域随意分散”。为了读取第 i 个元素，须从链头沿 `next` 走 $i-1$ 步，所以链表不支持按位序的常数时间随机访问。

已知有效结点 `p`，在其后插入新结点 `s`：先令 `s->next=p->next` 保存旧后继，再令 `p->next=s`。若先把 `p->next` 改成 `s`，随后从 `p->next` 取得的已不是旧后继，可能产生自环并丢失后半链。

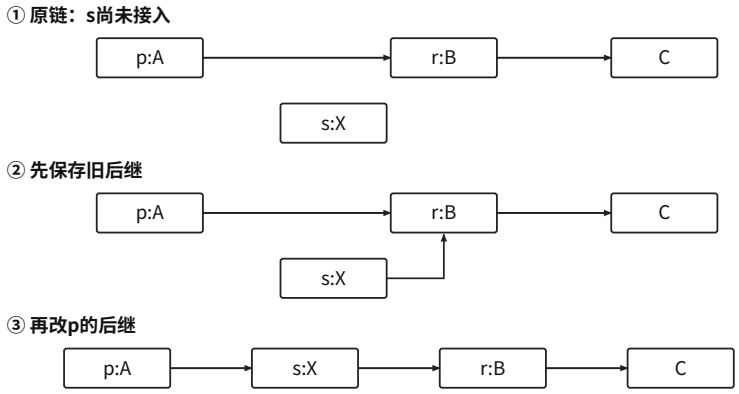


Figure 2.2: 插入前、保存旧后继、接入新结点；虚线表示待改链接

删除 `p` 的后继时，先检查 `p->next!=NULL`，再用 `q=p->next` 暂存待删结点，执行 `p->next=q->next`，最后释放 `q`。释放后不能继续读取 `q->next`。若只给待删结点的指针而不给其前驱，通常还要从头寻找前驱；用“复制后继数据再删后继”的办法不适用于尾结点，还可能改变题目要求保留的结点身份。

头插法不断把新结点接到头结点之后，输入次序会逆转；尾插法保存尾指针，每次接到尾部，能保留输入次序。两者若都维护所需指针，建立 n 个结点的链表均为 $O(n)$；若每插入一个元素都重新从头找尾部，才会累积到 $O(n^2)$。

2.4.1 原地逆置：未处理部分不能丢

维护 prev 为已逆置部分的新首结点，cur 为未处理部分的首结点。每次先暂存 nxt=cur->next，再把 cur->next 指向 prev，随后推进两个工作指针。

不变量·固定间隔保持到尾

fast 先走 k 步后，两指针始终相隔 k 条后继步。当 fast 到空位置时，slow 距空位置也恰为 k 步，故指向倒数第 k 个结点。先走阶段遇空可直接识别 k 超过表长。

```
void reverseList(LNode *L) {       // 带头结点
    LNode *prev=NULL, *cur=L->next;
    while (cur!=NULL) {
        LNode *nxt=cur->next;
        cur->next=prev;
        prev=cur;
        cur=nxt;
    }
    L->next=prev;
}
```

每轮只移动一个结点到逆置区。空表时循环不执行，头结点仍指向空；单结点时其 next 仍为空。时间 $\Theta(n)$，辅助空间 $\Theta(1)$。递归逆置虽然也能线性运行，却要保留递归栈，不能与迭代实现的空间混算。

2.4.2 例 2-2　如何一次扫描找到倒数第 k 个结点

例 2.4.1 (配套例题)

配套例题：已知一个带有表头结点的单链表，结点结构为 data | next，只给出了头指针 list。在不改变链表的前提下，设计尽可能高效的算法，查找倒数第 k 个位置上的结点（k 为正整数）。若成功，输出该结点的 data 值并返回 1；否则只返回 0。要求说明设计思想、实现步骤，并用 C、C++ 或 Java 描述算法，在关键处加注释。

解析。倒数位置由尾部决定，但链表不能直接向前访问。让两个指针保持固定间隔：fast 先从首元结点前进 k 步，再让 fast 与 slow 同步前进。若表长为 n，fast 到达空位置时共走了 n 步，slow 只走了 $n-k$ 步，正好指向第 $n-k+1$ 个结点。

```
int searchK(LNode *list, int k) {
    if (k<=0) return 0;
    LNode *fast=list->next, *slow=list->next;
    for (int j=0; j<k; ++j) {
        if (fast==NULL) return 0;   // 表长不足 k
        fast=fast->next;
    }
    while (fast!=NULL) {
        fast=fast->next;
        slow=slow->next;
    }
    printf("%d", slow->data);
    return 1;
}
```

以长度 5、$k=2$ 为例，先让 fast 指向第 3 个结点，slow 留在第 1 个；同步经过 “4 与 2—5 与 3—空与 4”，第 4 个即倒数第 2 个。$k=n$ 时第一阶段恰好走到空位置，第二阶段不执行，返回首元结点；$k>n$

时在第一阶段报失败。两个指针合计走 $O(n)$ 步，辅助空间 $O(1)$，没有修改任何链接。

最小反例·旧前驱必须还能取得

只有 r、p 两个相邻结点时，在 p 前插 q 的最终关系是 r↔q↔p。若第一句覆盖 p->prior，之后再从它读取就失去 r；四条目标链接全部成立才算插入完成。

2.5 双链表、循环链表与静态链表

双链表为每个结点增加 `prior`，以便从当前结点访问直接前驱。已知 `p` 后，删除 `p` 可在 $O(1)$ 时间内同时接好两侧；但这不消除按位序寻找 `p` 的成本。

2.5.1 例 2-3　把结点插到双链表结点之前

例 2.5.1 (配套例题)

〖2.3-015〗在双链表中向 `p` 所指的结点之前插入结点 `q`，正确操作是（　）。

```
A. p->prior=q; q->next=p;
   p->prior->next=q; q->prior=p->prior;
B. q->prior=p->prior; p->prior->next=q;
   q->next=p; p->prior=q->next;
C. q->next=p; p->next=q;
   q->prior->next=q; q->next=p;
D. p->prior->next=q; q->next=p;
   q->prior=p->prior; p->prior=q;
```

解析。假设 `p` 有有效前驱（可以是头结点）。记旧前驱为 `r`，结果必须同时满足 `r->next=q`、`q->prior=r`、`q->next=p`、`p->prior=q`。D 在最后才修改 `p->prior`，因此前三句还可以取得旧前驱，答案为 **D**。A 的第一句过早覆盖旧前驱；B 最后把 `p->prior` 设成了 `p` 自身；C 既改错方向，又使用尚未设置的 `q->prior`。

写程序时也可先 `q->prior=p->prior; q->next=p;`，再改两侧结点的链接。若 `p` 是不带头结点链表的首元结点，前驱为空，须单独更新头指针，不能直接解引用这个空前驱。

循环单链表把尾结点的 `next` 指回链头。带头结点时，空表满足 `L->next==L`；`L==NULL` 只说明没有有效头结点，并非这个模型的空表条件。遍历应在重新遇到头结点时结束，而不是等待 `NULL`。

只给尾指针 `rear` 的无头循环单链表，可以用 `rear->next` 取得首元结点；两个非空循环单链表头尾相接时，交换各尾部的后继链接即可完成 $O(1)$ 拼接。若只给头指针且没有尾链接，要先找到尾部，通常需要线性时间。若存在头结点，拼接时须处理多余头结点，不能把它当作数据留在两表之间。

循环双链表的头结点同时连接首尾，空表时 `next` 与 `prior` 都指回头结点。**静态链表**用数组保存结点，用整数下标代替机器指针；逻辑后继由游标决定，不要求结点下标相邻。还须维护空闲结点链或其他管理空闲位置的结构，数组开得大不等于每个位置都是有效数据。

2.6 双指针、归并与原地处理

错项排除·字符相同不等于共享

两个结点都存字符 i，也可能位于不同地址。公共后缀要求从某处起共享同一串结点，应比较指针而不是 data；共享链不能再按两条独占链分别释放。

这些方法把题目的约束变成指针之间的不变关系。使用前先确认能否修改原数据、是否有序、是否共享结点，以及是否需要保留重复元素。

2.6.1 例 2-4　数组循环左移怎样避免反复搬移

例 2.6.1 (配套例题)

配套例题：将 $n\ (n>1)$ 个整数存放在一维数组 R 中。设计一个时间、空间都尽可能高效的算法，将序列循环左移 $p\ (0<p<n)$ 个位置，即把 $(X_0,\dots,X_{n-1})$ 变为 $(X_p,\dots,X_{n-1},X_0,\dots,X_{p-1})$。给出思想、C/C++ 或 Java 实现，并说明时间与空间复杂度。

解析。题目要求两个连续区段交换位置，同时保持各段内部次序。设前 p 个元素为 A，其余为 B。分别逆置得 A^RB^R，再整体逆置，即得 BA。两个区段各被逆置两次，内部方向恢复，而相对位置交换。

```
void reverse(int R[], int l, int r) {
    while (l<r) {
        int t=R[l]; R[l]=R[r]; R[r]=t;
        ++l; --r;
    }
}
void rotateLeft(int R[], int n, int p) {
    reverse(R,0,p-1);
    reverse(R,p,n-1);
    reverse(R,0,n-1);
}
```

例如 `abcdefgh` 左移 3 位，依次得到 `cbadefgh`、`cbahgfed`、`defghabc`。总交换次数不超过 n，时间 $O(n)$；只用常数临时变量，空间 $O(1)$。若将首元素移到末尾重复 p 次，时间为 $O(np)$，当 p 与 n 同阶时会退化。三次逆置可处理 $p=1$ 和 $p=n-1$，无需另写搬移循环。

2.6.2 有序归并为什么可以一次扫描

两个递增表的当前首元素中，较小者一定是剩余全部元素中最小的。输出它并推进对应指针，重复直到某表为空，再接上另一个表的剩余部分。合并长度为 m,n 的两个表需要 $O(m+n)$ 时间；数组版本通常另开输出区，链表版本在允许破坏原表时可直接重接结点。

比较相等元素时，若希望保持原相对次序，先取左表元素。若题目要求集合并集而不是多重集合合并，则相等时只输出一次并推进两个指针；这项条件会改变算法，不能只改题目的答案。

2.6.3 例 2-5　相同后缀是相同字符还是相同结点

例 2.6.2 (配套例题)

配套例题：采用带头结点的单链表保存单词，两个单词可共享相同后缀的存储空间。例如 `loading` 与 `being` 共享从字符 `i` 开始的结点。`str1`、`str2` 指向两个头结点，结点为 `data | next`。设计时间上尽可能高效的算法，找到共同后缀的起始结点，给出思想、

程序及时间复杂度。

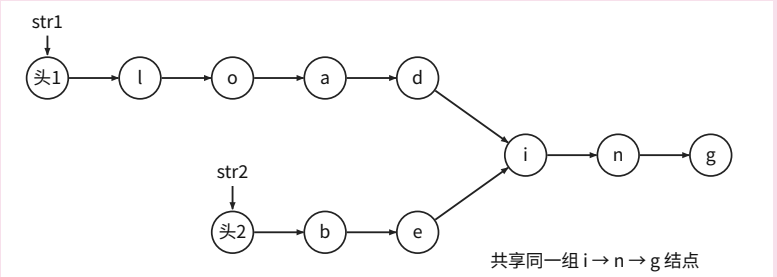


Figure 2.3: 两个前缀分别连接到同一个 i 结点，随后共用 n、g 结点

最小反例·剩下候选仍需复核

序列 1、2、3 经抵消可留下 3，但 3 只出现一次，未超过一半。抵消给出的是存在多数时的必要候选，并不证明多数存在；第二次计数补上充分性检验。

解析。共享指的是地址相同，不能以 `p->data==q->data` 判断。单链表一旦在某结点汇合，其后继也相同，所以两表从公共起点到末尾的长度相等。先各扫描一次求长 m, n，让较长表先走 $|m-n|$ 步，两指针便处于距表尾相同的位置；此后同步前进，第一次地址相等的位置就是答案。

```
LNode *commonSuffix(LNode *str1, LNode *str2) {
    int m=0,n=0;
    for (LNode *p=str1->next; p; p=p->next) ++m;
    for (LNode *q=str2->next; q; q=q->next) ++n;
    LNode *p=str1->next, *q=str2->next;
    while (m>n) { p=p->next; --m; }
    while (n>m) { q=q->next; --n; }
    while (p!=q) { p=p->next; q=q->next; }
    return p;                    // 无公共结点时二者同时为 NULL
}
```

图中两表长度分别为 7 和 5，先让长表跨过 `l,o`，再比较 `a` 与 `b`、`d` 与 `e`，最后到达同一个 `i`。若两表内容相同却各自分配结点，则直到空位置才相等，返回空。总时间 $O(m+n)$，辅助空间 $O(1)$，适用于有限、无环单链表。

2.6.4 例 2-6　主元素为什么还要再计数

例 2.6.3 (配套例题)

配套例题：整数序列 $A=(a_0,\ldots,a_{n-1})$ 满足 $0\le a_i<n$。某值出现次数超过 $n/2$ 时，称它为主元素。例如 $(0,5,5,3,5,7,5,5)$ 的主元素为 5，而 $(0,5,5,3,5,1,5,7)$ 没有主元素。设计尽可能高效的算法，存在主元素时输出该值，否则输出 -1；给出思想、程序及时间、空间复杂度。

解析。每次删除两个值不同的元素，不会消灭一个原本占严格多数的值：若删去一个多数元素和一个其他元素，多数仍超过剩余的一半；若删去两个其他元素，多数优势更大。用一个候选值和计数模拟这种抵消。计数为 0 时选新候选；相同则加 1，不同则减 1。

```
int majority(int A[], int n) {
    int candidate=0, count=0;
    for (int i=0; i<n; ++i) {
        if (count==0) { candidate=A[i]; count=1; }
        else if (A[i]==candidate) ++count;
        else --count;
    }
    int actual=0;
    for (int i=0; i<n; ++i)
        if (A[i]==candidate) ++actual;
    return actual>n/2 ? candidate : -1;
}
```

不变量·头结点不随表空而消失

有效头结点始终存在，数据结点从 L->next 开始。空表并不意味着 L 为空；统一前驱定位时 pos=0 对应头结点，才能让第一个位置的插入与其他位置共用同样两步链接。

抵消只保证“若有主元素，最后候选一定是它”，没有保证主元素存在。例如 `1,2,3` 最后仍可能留下候选 3，它只出现一次，所以必须第二次扫描核实。两个线性扫描合计 $O(n)$ 时间，额外空间 $O(1)$。题目给出的值域也允许用计数数组，但需 $O(n)$ 辅助空间；此处没有必要为省下一次扫描而付出这部分空间。

2.7 单链表的类型、定位与修改

教材用 `LNode` 表示结点类型，用 `LinkList` 表示指向结点的指针类型。`LNode *`与 `LinkList` 在这里是同一种指针类型，名称强调的语义不同：前者常强调某个结点，后者常强调一张表的头指针。

```
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    int j = 0;
    LNode *p = L;
    while (p != NULL && j < i) {
        p = p->next;
        ++j;
    }
    return p;
}
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) p = p->next;
    return p;
}
```

以上采用有效头结点，`GetElem(L,0)`返回头结点，`GetElem(L,i)`在 i 超过表长时返回空。按值查找从首元结点开始。求表长可令计数初值 0，沿 `L->next` 逐结点递增，不能把头结点算作有效元素。

2.7.1 后插、按位插入与删除后继

后插的前提是 p 指向有效结点。申请成功后，先保存原后继再接入；两次指针改写的次序决定原链是否保留。

```cpp
bool InsertNextNode(LNode *p, ElemType e) {
    if (p == NULL) return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) return false;
    s->data = e;
    s->next = p->next;
    p->next = s;
    return true;
}
bool ListInsert(LinkList &L, int i, ElemType e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    return InsertNextNode(p, e);
}
bool ListDelete(LinkList &L, int i, ElemType &e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    if (p == NULL || p->next == NULL) return false;
    LNode *q = p->next;
    e = q->data;
    p->next = q->next;
    free(q);
    return true;
}
```

辨析·第一位改谁

不带头结点时，第 1 位插入与删除要修改 L 本身；引用参数把新首元地址写回调用者。

`ListInsert` 还要定位第 i−1 个结点，因此按位序操作最坏 $O(n)$；已知前驱后的局部操作才是 $O(1)$。删除 q 前保存其后继并接回链，释放后不再读取 q。独占链销毁时依次保存后继、释放当前结点，最后释放头结点；两个共享后缀的表不能分别完整释放同一批共享结点。

只给结点 p、希望在 p 前插入新值时，普通方法先找前驱。若题目允许改变 p 的数据，可将新结点插在 p 后并复制旧数据，再把新值写入 p，常数时间得到相同的值序列；这种方法不保留 “哪个值属于哪个结点” 的身份。只给 p 删除自身时也可复制后继数据再删后继，但 p 为尾结点时失效。

2.7.2 不带头结点时，第一位需要单独处理

不带头结点的空表满足 `L==NULL`。在位置 1 插入要改头指针，因此接口用引用参数；其余位置先找前驱。

```cpp
bool ListInsert_NoHead(LinkList &L, int i, ElemType e) {
    if (i < 1) return false;
    if (i == 1) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        if (s == NULL) return false;
        s->data = e;
        s->next = L;
        L = s;
        return true;
    }
    LNode *p = L;
    int j = 1;
    while (p != NULL && j < i - 1) {
        p = p->next; ++j;
    }
    return InsertNextNode(p, e);
}
```

不带头结点删除首元时同样要令 `L=L->next` 并释放旧首元；带头结点则统一通过头结点的后继处理。头指针是一个变量，头结点是一个哨兵对象，首元结点是第一个实际数据对象，三者不能互换。

辨析·尾插不能反复找尾

r 始终指最后一个结点，每轮直接接入，总时间线性；每轮从头找尾才会累计到平方阶。

2.8 建表：头插逆序，尾插保持输入次序

头插法每次把新元素接到头结点之后，因此输入 a、b、c 后，数据次序为 c、b、a。尾插法维护尾指针 r，始终指向当前最后一个结点，输入同样数据后次序为 a、b、c。

```
LinkList List_HeadInsert(LinkList &L) {
    LNode *s;
    int x;
    L = (LinkList)malloc(sizeof(LNode));
    L->next = NULL;
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = L->next;
        L->next = s;
        scanf("%d", &x);
    }
    return L;
}
LinkList List_TailInsert(LinkList &L) {
    int x;
    L = (LinkList)malloc(sizeof(LNode));
    LNode *s, *r = L;
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        r->next = s;
        r = s;
        scanf("%d", &x);
    }
    r->next = NULL;
    return L;
}
```

两段采用教材的返回头指针接口，以输入读取和 malloc 均成功为前提，输入 9999 结束，因此 9999 不能同时作为这一输入协议中的普通数据。若首次输入就是 9999，头插法通过 L->next=NULL 得到空表，尾插法通过循环后的 r->next=NULL 得到空表。实际程序还应检查 scanf 返回值和申请结果；失败时须保存并释放已建前缀。两种建表均为 $O(n)$；尾插每次若重新从头找尾部，才会变成 $O(n^2)$。

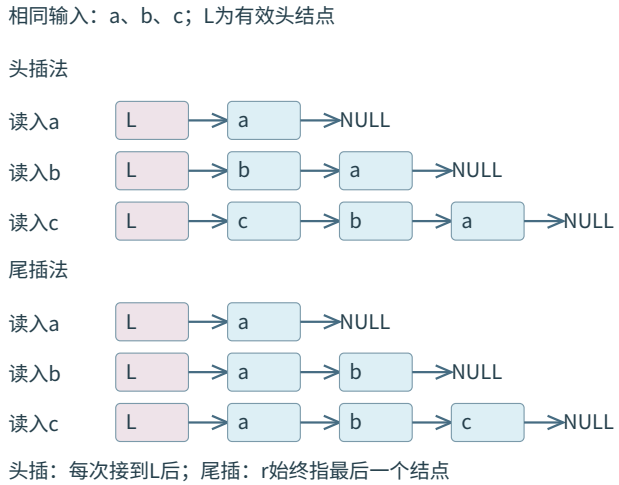


Figure 2.4: 同样输入 a、b、c，头插与尾插的中间状态和最终次序

辨析·循环终止条件

普通单链表等待 NULL，循环表等待回到头结点。带头循环表空时头结点的 next 指向自身。

2.9 双链表的局部操作与循环边界

双链结点同时保存前驱和后继。以下 p 有有效后继时采用统一接入；非循环链尾的后继为空，需对后继的 prior 赋值另作检查。

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinklist;
bool InsertNextDNode(DNode *p, DNode *s) {
    if (p == NULL || s == NULL) return false;
    s->next = p->next;
    if (p->next != NULL) p->next->prior = s;
    s->prior = p;
    p->next = s;
    return true;
}
bool DeleteNextDNode(DNode *p) {
    if (p == NULL || p->next == NULL) return false;
    DNode *q = p->next;
    p->next = q->next;
    if (q->next != NULL) q->next->prior = p;
    free(q);
    return true;
}
```

带头循环双链表空时 `L->next==L` 且 `L->prior==L`；数据结点的前后方向均可遍历，但遇头结点时要停止，不能把头结点当作普通数据删除。循环单链表查尾条件是 `p->next==L`，与普通单链表的 `p->next==NULL` 不同。只用头指针时找尾仍需遍历；保存尾指针后可通过 `rear->next` 找到头部。

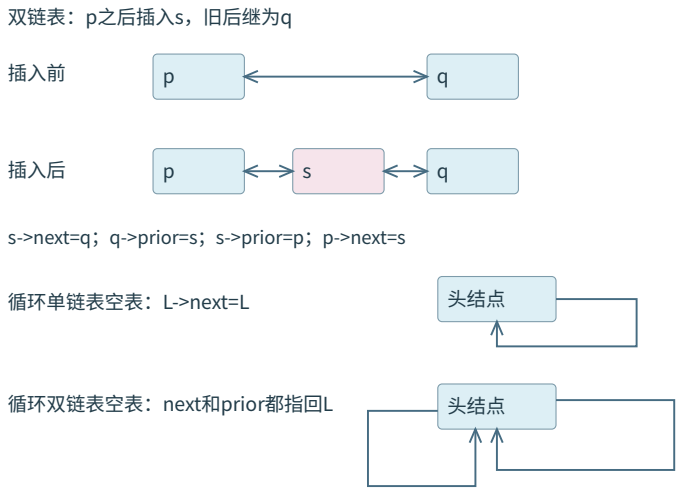


Figure 2.5: 双链表插入前后及两种循环链表的空状态

课程连接·表示影响访问局部性

连续数组通常能利用缓存中的相邻数据，分散结点可能增加缓存未命中。渐近复杂度相同也不保证实际用时相同；比较表示时同时检查数据规模、访问方式和存储开销。

静态链表以数组下标充当游标。例如 `struct { ElemType data; int next; } SLinkList[MaxSize];`中，next 记录的是下一个结点占用的数组位置，并非机器地址。还需要管理空闲位置；课件的一种具体表示令 0 号结点为头结点，next=-1 表示链尾，next=-2 表示该数组位置空闲：初始化令 a[0].next=-1，其余位置 next=-2；插入先找空位再接游标，删除先接回前驱游标，再将被删位置标为空闲。结束和空闲是两个不同状态。静态链表仍按链接顺序定位，不因结点集中在数组就获得按逻辑位序随机访问。

课件的静态链表把同一个逻辑序列 e1、e2、e3、e4 分散在下列数组位置：

数组下标	data	next
0	头结点	2
1	e2	6
2	e1	1
3	e4	-1
4、5	空闲	-2
6	e3	3
7、8、9	空闲	-2

遍历游标顺序为 $0\to2\to1\to6\to3\to-1$。若每结点数据域 4B、游标 4B，数组首地址为 addr，则 e1 位于 addr+2×8；它是逻辑第 1 个元素，物理上却占数组第 2 号结点。

2.10 结构选择与操作代价对照

操作条件	顺序表	单链表（有效头结点）	选择时还需检查
按位序取值	$O(1)$	$O(n)$	是否要求随机访问
无序按值查找	$O(n)$	$O(n)$	值是否有序、是否另有索引
按位序插入/删除	最坏 $O(n)$	定位最坏 $O(n)$	是否已给有效前驱
已知前驱后的局部修改	仍可能搬移后缀	$O(1)$	申请/释放与结点身份
存储开销	连续容量可能有空闲	每结点含链接域	局部性、碎片和最大规模

顺序表逻辑相邻通常也物理相邻，有利于连续访问；链表用额外链接换取接入灵活性，不能由此得出“链表一定更省空间”。静态链表虽然使用数组分配结点，逻辑后继仍由游标指定，所以不具备按逻辑位序直接寻址的能力。

不变量·删后检查新后继

若当前前驱后连续两个结点都等于 x，删掉第一个后马上前进，就可能跳过第二个。前驱只在保留结点时前进，才能让每个新后继都得到检查。

2.11 连续删除与有序去重的线性写法

> **删除后留在前驱，保留后才前进**
>
> 删除单链表所有值为 x 的结点时，用 p 指向已处理部分的最后一个保留结点。若 p 后继应删除，就摘除后继续检查 p 的新后继；否则 p 向前走。相邻多个 x 正是不能在每次删除后都移动 p 的原因。

```
void eraseAll(LNode* L, int x) {
    LNode* p = L;
    while (p->next != NULL) {
        if (p->next->data == x) {
            LNode* q = p->next;
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

顺序表有序去重可让读指针从左到右扫描，写指针只扩展“已保留的互异前缀”。若当前值不同于最后一个保留值才写入；每个元素只处理常数次，总时间 $O(n)$，额外空间 $O(1)$。未排序数组中，相邻不重复不能保证全局不重复，不能悄悄去掉有序条件。

2.12 主动回忆自测

以下为原创补充，检查边界和不变式。

1. 允许修改单链表，要求删除所有值为 x 的结点。删除一个结点后，指向前驱的指针是否应立即后移?
2. 两个无环单链表长度相同，尾结点地址相同，能否断定它们一定有公共后缀?
3. 顺序表去除相邻重复元素时，为什么可用“已保留长度”作为写指针，而不必每删一个元素都搬移后半表?

答案。1. 不应立即后移；新的后继还可能等于 x，应继续检查当前前驱的后继。只有保留一个结点时前驱才推进。2. 能；至少尾结点本身就是公共后缀，公共起点还可能更早。3. 扫描指针只读未处理区，写指针只向前形成已保留区；每个元素最多被检查和写入常数次，总时间 $O(n)$。若原表无序，这只能消除相邻重复，不能保证全表唯一。

2.12.1 补充自拟回忆题

从边界恢复算法

4. 长度 n 的顺序表允许在哪些位序插入，在哪些位序删除？各自等概率模型的分母是什么？
5. 有效头结点、头指针、首元结点如何区分？带头结点空表中哪一个为空？
6. 只给单链表尾结点指针，能否总在 O(1) 时间内删除该结点并保留其他结点身份？
7. 主元素候选的计数归零意味着此前完全没有主元素吗？为什么最后还要核计数？
8. 两个内容相同却分别分配的单链表，公共后缀算法返回什么？适用前提是什么？

补充题核对答案

4. 插入位序 1 到 n+1，删除 1 到 n；平均插入移动数除以 n+1，平均删除移动数除以 n。
5. 头指针是地址变量，头结点是哨兵对象，首元结点是第一个有效元素。有效头结点存在时，空表是 L->next 为空。
6. 不能。单链表尾结点无后继可复制，且没有直接前驱链接，通常要从头找前驱。
7. 不意味着不存在严格多数；抵消保证 “若存在，最后候选是它”，不能反推候选一定占多数，必须再次扫描。
8. 返回 NULL。地址相等才表示共享，前提是有限无环单链表且头结点和链接有效。

自查提示·区分定位与修改

已知结点时的局部操作与从头查找位置的总代价应分别计算。